

both urllib and urllib2 are accessed for accomplishing tasks. In spite of using both these modules, there are various drawbacks:

- First and foremost, it is unclear which module - urllib or urllib2 - is to be used and this is confusing, especially for beginners.
- In spite of urllib2 being an improved module, it does not provide all the functionalities.
- The documentation for both urllib and urllib2 is extremely difficult to understand and is heavily over-engineered.
- Even for a simple GET request, it is impossible to write a short script in Python using urllib2.

Here is a sample of the code required to perform a simple login:

```
import urllib
import urllib2
import re
import cookielib

jar = cookielib.FileCookieJar("cookie")
opener = urllib2.build_opener(urllib2.
HTTPCookieProcessor(jar))

url = 'http://example.com/login.php'
user_agent = 'Mozilla/4.0 (compatible; MSIE 5.5; Windows NT)'

data =
{
    "Submit": " ",
    "username": "x",
    "password": "x",
}

data = urllib.urlencode(data)
login_request = urllib2.Request(url, data)
login_reply = opener.open(login_request)
login_reply_data = login_reply.read()

login_success_msg = re.compile("Login Successful")

if login_success_msg.search(login_reply_data) is not None:
#Procede
else:
Print "Check whether you have given the right credentials"
```

Performing a simple login operation requires importing four different modules and writing large volumes of complex code. HTTP is simple and so should the scripts that access it be. Hence, it is vital to develop simpler and more efficient modules for accessing the Web when using Python.

Python Requests

'Requests' is a simple, easy-to-use HTTP library written in Python. The lead developer is Kenneth Reitz, who is also

a member of the Python Software Foundation. The current version is 2.2.1, released on January 23, 2014, and it is compatible with Python versions 2.6.8 and above. Requests makes interacting with Web services seamless, and it overcomes most of the difficulties faced with urllib/urllib2.

Installation: Installing Python Requests is very simple and can be done using any of the two methods mentioned below:

- *Pip:* This works with Python versions 2.6, 2.7, 3.1, 3.2 and 3.3:

```
$ pip install requests
$ sudo apt-get install python-pip
or
$ sudo apt-get install python-pip python-dev build-essential
```

- *Download from the source code:*

```
$ sudo apt-get install build-essential libncursesw5-dev
libreadline5-dev libssl-dev libgdbm-dev libc6-dev libsqlite3-
dev tk-dev
$ wget http://www.python.org/ftp/python/3.x/Python-3.x.tar.bz2
$ tar -xjf Python-3.xtar.bz2 cd Python-3.x
$ ./configure --prefix=/opt/python3
$ make
$ sudo make install
```

Parsing JSON

JSON is JavaScript Object Notation and is used for transmitting data between client and server. It is often found that Web pages have JSON embedded in their code. Hence, while receiving requests, we often get a response in the JSON format, which needs to be decoded. Python Requests have a built-in JSON decoder, which helps in parsing JSON code. You can just import the JSON module in your code.

How to know if the response is in the JSON format: After making a get request there is a response object 'r' from which we can get information like the status code, header etc.

```
import requests

r = requests.get("http://www.example.com")
print r.status_code
print r.headers['content-type']
```

```
Output:
200
'application/json'
```

If the content-type in the header is of the type 'application/json' then the response is in the JSON format.

How to parse using the JSON built-in module and Requests:

- `json.load(response)` is used for decoding the response